

Keywords: memory-efficient fine-tuning • operator-norm optimization • ternary update • LLM training

TACO: Ternary Absolute-max Column-wise One-sparse Optimizer for LLM Fine-Tuning

174× Less Optimizer State Than AdamW8bit, Full-Parameter Fine-Tuning of 32B Models on One 80 GB H100, Comparable Accuracy

By Jichao Jiang, Cristian McGee, El Houcine Bergou, Hanqin Cai & Aritra Dutta (University of Central Florida; Mohammed VI Polytechnic University)

arXiv:2610.02199 • October 2026

TACO takes the operator-norm steepest-descent view to its logical extreme: it selects the sign of the largest-magnitude entry in each column of every two-dimensional weight matrix—a ternary, one-sparse-per-column update that requires essentially zero persistent optimizer state—and reduces AdamW8bit’s 27.7 GB optimizer footprint on OPT-13B to 0.16 GB while matching downstream accuracy across multiple model families and enabling 32B-scale full-parameter fine-tuning on a single H100.

AT A GLANCE

Problem: AdamW and quantized variants require 2–3× the model size in optimizer state, limiting full-parameter fine-tuning to small models on a single GPU.

Why it matters: Full-parameter fine-tuning outperforms LoRA on complex domain adaptation; enabling it for 30B+ models democratizes access.

Method: Exact steepest descent under dimension-normalized 1→1 operator norm selects sign of the maximum-magnitude column entry; no persistent momentum buffers.

Result: 174× optimizer state reduction; 2.9× peak memory reduction; accuracy within 0.5–1.0% of AdamW on SuperGLUE, SQuAD v2.

Evidence: OPT, LLaMA, Mistral, Phi families; fine-tuning tasks including SuperGLUE, SQuAD v2, and instruction following.

The Big Picture

Full-parameter fine-tuning of large language models is the gold standard for domain adaptation: it updates every weight, avoids the rank constraints of parameter-efficient methods (LoRA, prefix tuning), and captures distribution shifts that low-rank approximations cannot

represent. Yet it has been practically out of reach for models above ~7B parameters on a single 80 GB GPU because AdamW—the dominant optimizer—maintains first and second moment buffers that add 2× the model parameter count in optimizer state.

TACO addresses this by rethinking what state an optimizer actually needs. Drawing on the operator-norm steepest-descent perspective introduced by Muon, TACO shows that the exact optimal update under a dimension-normalized 1→1 operator norm requires no momentum buffers whatsoever—only the current-step gradient, which is discarded immediately after use.

Why Existing Approaches Fall Short

AdamW: Two full-precision moment buffers (first and second order) add 2× the model’s FP32 parameter count to GPU memory. On OPT-13B this is 27.7 GB of persistent state alone.

AdamW8bit: Quantizes moment buffers to 8 bits, cutting state to $\approx 27.7 \text{ GB}/2 = 13.9 \text{ GB}$ —a useful reduction but still far larger than TACO’s 0.16 GB.

SignSGD: Takes the sign of the full gradient column-by-column but does not arise from a principled operator-norm steepest-descent argument, and can exhibit slow convergence on poorly scaled gradients.

Muon: Derives updates via operator-norm steepest descent and applies Nesterov momentum, but still maintains a first-moment buffer.

LoRA: Reduces state by fine-tuning only low-rank adapters, but cannot update all parameters and underperforms full-rank methods on complex adaptation.

Core Mechanism

Operator-norm steepest descent. Given a weight matrix $W \in \mathbb{R}^{m \times n}$ with gradient G , the steepest-descent direction under the dimension-normalized $1 \rightarrow 1$ operator norm constraint selects, for each column j , the row index achieving the maximum gradient magnitude, and assigns a ternary entry equal to the sign of that element.

Formally, the update matrix $U \in \{-1, 0, +1\}^{m \times n}$ has exactly one nonzero entry per column—hence “one-sparse per column” and “ternary”. No previous-step information is accumulated: the only state at any step is the current gradient, discarded immediately after U is computed.

Technical Formulation

For each column $j \in [n]$ of gradient $G^{(t)}$:

$$i^*(j) = \arg \max_{i \in [m]} |G_{ij}^{(t)}|$$

$$U_{ij}^{(t)} = \begin{cases} \text{sign}\left(G_{i^*(j),j}^{(t)}\right) & i = i^*(j) \\ 0 & \text{otherwise} \end{cases}$$

Weight update:

$$W^{(t+1)} = W^{(t)} - \eta U^{(t)}$$

Persistent optimizer state between steps: $\emptyset$ (the BF16 gradient buffer $G^{(t)}$ is freed immediately after the update).

Under a convex loss with Lipschitz gradient, TACO achieves $O(1/\sqrt{T})$ convergence matching signSGD, while the update $U^{(t)}$ is the exact minimizer of the linearized objective subject to the unit operator-norm ball.

Learning or Inference Procedure

Forward: Standard.

Backward: Compute gradient $G^{(t)}$ for each 2D weight matrix.

Update (per-matrix):

1. Compute argmax over each column of $|G^{(t)}|$.
2. Construct ternary one-sparse matrix $U^{(t)}$.
3. Apply $W \leftarrow W - \eta U^{(t)}$ in-place.
4. Free $G^{(t)}$.

Non-2D tensors (embeddings, 1D biases) fall back to signSGD. Learning rate: cosine or linear warmup-decay schedule; no additional hyperparameters beyond η .

What the Guarantee Says

For convex losses, TACO converges at rate $O(1/\sqrt{T})$, matching signSGD. The $U^{(t)}$ is the exact steepest-descent direction under the $1 \rightarrow 1$ operator norm: no other ternary update achieves a greater linearized objective decrease under the operator-norm geometry. The persistent state between steps is strictly zero beyond the model weights, providing a hard lower bound on optimizer memory that no further quantization can improve.

Experimental Findings

- **Optimizer state:** 27.7 GB (AdamW8bit) $\rightarrow$ 0.16 GB (TACO) on OPT-13B: a $174\times$ reduction.
- **Peak training memory:** 80.6 GB $\rightarrow$ 27.5 GB on OPT-13B: a $2.9\times$ reduction.
- **Large-model access:** OPT-30B and LLaMA-32B can be fully fine-tuned on a single 80 GB H100—previously impossible with AdamW.
- **Accuracy:** Within 0.5–1.0% of AdamW on SuperGLUE, SQuAD v2, and instruction following, across OPT, LLaMA, Mistral, Phi.
- **Throughput:** Training step runtime within 5% of AdamW8bit owing to the simplicity of the argmax-and-sign update.

Ablations and Interpretation

Row-wise vs. column-wise argmax: Row-wise argmax (violating the operator-norm derivation) slightly degrades accuracy, confirming the theoretical motivation matters in practice.

Top- k per column (TACO- k): Allowing $k > 1$ nonzero entries per column interpolates between TACO and a dense gradient, recovering AdamW accuracy at the cost of k -fold increased state; $k = 1$ is Pareto-optimal for memory-constrained settings.

Momentum ablation: Adding a first-moment moving average of $U^{(t)}$ improves convergence speed on some tasks but doubles persistent state; omitting momentum is preferred when memory is the bottleneck.

TACO vs. LoRA: At equal GPU memory budget, full-parameter TACO outperforms LoRA on domain-specific tasks, confirming the value of full-rank updates when memory constraints are relaxed by the optimizer.

Reference

Jichao Jiang, Cristian McGee, El Houcine Bergou, Hanqin Cai, Aritra Dutta. **TACO: Ternary Absolute-max Column-wise One-sparse Optimizer for LLM Fine-Tuning**. arXiv:2610.02199, October 2026.
<https://arxiv.org/abs/2610.02199>